Asynchronous Exception handling in Haskell: https://www.fpcomplete.com/blog/2018/04/async-exception-handling-haskell/
Control.Exception: https://hackage.haskell.org/package/base/docs/Control-Exception.html

Resources
[FP Complete] Safe exception handling tutorial: https://www.fpcomplete.com/haskell/tutorial/exceptions/
[FP Complete] Asynchronous exceptions: https://www.fpcomplete.com/blog/2018/04/async-exception-handling-haskell/
[FP Complete] Exceptions best practices: https://www.fpcomplete.com/blog/2016/11/exceptions-best-practices-haskell/

```
data SomeException = forall e. Exception e =>
SomeException e

class (Typeable e, Show e) => Exception e
  toException :: e -> SomeException
  fromException :: SomeException -> Maybe e
  displayException :: e -> String

data IOException
data ArithException = Overflow | Underflow |
LossOfPrecision | DivideByZero | Denormal |
RatioZeroDenominator
data ArrayException = IndexOutOfBounds String |
UndefinedElement String
data NonTermintion = NonTermination
data NestedAtomically = NestedAtomically
data ErrorCall = ErrorCallWithLocation String String
                     -- error thrown by error function
data TypeError = TypeError String
...

ioError :: IOError -> IO a

throwIO :: Exception e => e -> IO a
catch :: Exception e => IO a -> (e -> IO a) -> IO a
catches :: Exception e => IO a -> [Handler a] -> IO a
catchJust :: Exception e => (e -> Maybe e') -> IO a ->
(e' -> IO a) -> IO a      -- select exception subtype e'.
Rethrow otherwise.
handle :: Exception e => (e -> IO a) -> IO a -> IO a
                          -- catch reversed
handleJust :: Exception e => (e -> Maybe e') -> (e' ->
IO a) -> IO a -> IO a

try :: Exception e => IO a -> IO (Either e a)
                          -- catch with Either
tryJust :: Exception e => (e -> Maybe e') -> IO a -> IO
(Either e' a)

bracket :: IO a -> (a -> IO b) -> (a -> IO c) -> IO c
bracket :: IO a -> IO b -> IO c -> IO c
```

```
bracketOnError :: IO a -> (a -> IO b) -> (a -> IO c) ->
IO c
finally :: IO a -> IO b -> IO a
onException :: IO a -> IO b -> IO a
```

Impure exceptions

```
-- Control Exception
newtype AssertionFailed = AssertionFailed String
throw :: forall (r :: RuntimeRep). forall (a :: Type r).
forall e. Exception e => e -> a
assert :: Bool -> a -> a
evaluate :: a -> IO a
```

SomeException type
The **root exception** type.

```
data SomeException = forall e. Exception e =>
SomeException e
```

e.g. can hold any exception type

```
x :: SomeException
x = SomeException DivideByZero        -- SomeException
with (DivideByZero::ArithException)
x = SomeException StackOverflow       -- SomeException
with (StackOverflow::AsyncException)
```

Exceptions
3 types of exceptions
1) **imprecise** exceptions: error thrown in **pure** code
Control.Exception: https://hackage.haskell.org/package/base/docs/Control-Exception.html#g:8
A semantics for imprecise exceptions: https://www.microsoft.com/en-us/research/wp-content/uploads/1999/05/except.pdf

Discouraged since error may have **unexpected** behavior. e.g. may not be caught
e.g.

```
error "BAD"                -- Exception: BAD
undefined                  -- Exception:
Prelude.undefined
throw DivideByZero         -- Exception: divide by zero
head []                    -- Exception: Prelude.head:
empty list
```

2) **synchronous** exceptions: error thrown in **IO** in the **main thread**.
e.g.

```
do
```

```
putStrLn "Test"
throwIO DivideByZero
```

3) **asynchronous** exceptions
Control.Exception: https://hackage.haskell.org/package/base/docs/Control-Exception.html#g:10
An exception thrown into a target thread from a **different thread** to **interrupt** the target thread
e.g. timeout
See below for details

***) Usage: define and throw custom error**
Just need to derive **Exception class** with default implementation.

e.g.

```
import Control.Exception
-- step: define error
data MyError = DivZero | AnotherError deriving (Show)
instance Exception MyError
where                                   -- can be empty
i.e. default implementation

-- step: throw IO error
divPureIO :: Int -> Int -> IO Int
divPureIO _ 0 = throwIO DivZero
divPureIO x y = pure $ divPure x y

test :: IO ()
test = do
  x <- divPureIO 2 1             -- OK
  x <- divPureIO 2 0             -- exception DivZero

  -- e.g. catch
  x <- catch (divPureIO 2 0)                    -- x =
0. Prints "DivZero"
       (\(e::MyError) -> print e >> return 0)
```

e.g. same with DeriveAnyClass

```
{-# LANGUAGE DeriveAnyClass #-}
data MyError = DivZero | AnotherError
   deriving (Show, Exception)

-- with explicit strategies
{-# LANGUAGE DerivingStrategies #-}
data MyError = DivZero | AnotherError
   deriving stock Show
   deriving anyclass Exception
```

***) Usage: Exception hierarchy**
GHC.Exception.Type: https://hackage.haskell.org/package/base/docs/GHC-Exception-Type.html

Every exception is a **SomeException** by default.
Allow **catching** any **child exceptions**.
e.g.
SomeException > MyExceptionA > MySubException
> MyExceptionB

throwIO DivideByZero SomeException DivideByZero
throwIO MyExceptionB SomeException MyExceptionB
throwIO (MyExceptionA DivideByZero) SomeException (MyExceptionA DivideByZero)
throwIO MySubException SomeException (MyExceptionA MySubException)

catch @SomeException ... $ \(SomeException e) -> ... catches any of e = (MyExceptionA MySubException), (MyExceptionA e), MyExceptionB
catch @MyExceptionB ... $ \MyExceptionB -> ... MyExceptionB only
catch @MyExceptionA ... $ \(MExceptionA e) -> ... (MyExceptionA e) or (MyExceptionA MySubException)
catch @MySubException ... $ \MySubException -> ... MySubException only

e.g.

```
import Control.Exception
import Data.Data (cast0

data MyExceptionA = forall e. (Exception e) =>
MyExceptionA e

instance Show MyExceptionA where
  show (MyExceptionA e) = "MyExceptionA " <> show e

-- | SomeException (MyExceptionA e)
instance Exception MyExceptionA
                  -- SomeException > MyExceptionA

data MyExceptionB = MyExceptionB deriving (Eq, Show)

-- | SomeException (MyExceptionB e)
instance Exception MyExceptionB
                  -- SomeException > MyExceptionB

data MySubExceptionA = MySubExceptionA deriving (Eq,
Show)

-- | SomeException (MyExceptionA (MySubExceptionA e))
                  -- SomeException > MyExceptionA >
MySubExceptionA
instance Exception MySubExceptionA where
```

```
  toException = toException . MyExceptionA
  fromException e = do
    MyExceptionA e' <- fromException e
    cast e'
```

throw/catch

```
-- catch any
try @SomeException @Int (throwIO MySubExceptionA)
             -- Left (SomeException (MyExceptionA
MySubExceptionA)0
try @SomeException @Int (throwIO $ MyExceptionA
DivideByZero)         -- Left
(SomeException( MyExceptionA MySubExceptionA))
try @SomeException @Int (throwIO MyExceptionB)
             -- Left (SomeException MyExceptionB)

-- catch MyExceptionB only
try @MyExceptionB @Int (throwIO MySubExceptionA)
             -- uncaught (SomeException (MyExceptionA
MySubExceptionA))
try @MyExceptionB @Int (throwIO $ MyExceptionA
DivideByZero)         -- uncaught (SomeException
(MyExceptionA DivideByZero))
try @MyExceptionB @Int (throwIO MyExceptionB)
             -- Left MyExceptionB

-- catch any MyExceptionA
try @MyExceptionA @Int (throwIO MySubExceptionA)
             -- Left (MyExceptionA MySubException)
try @MyExceptionA @Int (throwIO $ MyExceptionA
DivideByZero)         -- Left (MyExceptionA DivideByZero)
try @MyExceptionA @Int (throwIO MyExceptionB)
             -- uncaught (SomeException MyExceptionB)

-- catch  MySubExceptionA only
try @MySubExceptionA @Int (throwIO MySubExceptionA)
             -- Left (MySubExceptionA)
try @MySubExceptionA @Int (throwIO $ MyExceptionA
DivideByZero)         -- uncaught (SomeException
(MyExceptionA DivideByZero))
try @MySubExceptionA @Int (throwIO MyExceptionB)
             -- uncaught (SomeException MyExceptionB)
```

GOTCHA: strange behavior of imprecise exceptions
https://www.microsoft.com/en-us/research/wp-content/uploads/1999/05/except.pdf

Imprecise exception semantics
If there is more than 1 error, the program may throw **either** error.
e.g.

```
x, y :: Int
```

```
x = error "x"
y = error "y"

x + y `seq` ()   -- may be error "x" OR error "y"
```

GOTCHA: uncaught exceptions in pure functions i.e. imprecise exceptions
Ref: https://hackage.haskell.org/package/base/docs/Control-Exception.html#g:8

Problem
Exception handlers like catch only catch exceptions **thrown in IO**.
i.e. **imprecise exceptions** may not be caught because the exception may have been evaluated **outside of IO**

e.g.

```
import Control.Exception

x = 1 `div` 0                    -- May be evaluated
outside of IO.

test :: IO ()
test = do
  catch @SomeException (return x) print        -- BAD.
Not caught.
  catch @SomeException (evaluate x) print      -- OK.
Caught.
  catch @ErrorCall (evaluate x) print          -- OK.
Catch imprecise only.
```

Solution
Imprecise exception: https://hackage.haskell.org/package/base/docs/Control-Exception.html
Pure code exceptions are called **imprecise** exceptions.
Imprecise exceptions may NOT be caught since they may be evaluated **outside of IO**.
In fact, imprecise exceptions may

In general, pure code SHOULD NOT have exceptions.
i.e.
1) do not **throw** imprecise exceptions
e.g. use Maybe/Either instead.
2) do not try to **catch** imprecise exceptions
Code SHOULD NOT have imprecise exceptions. If an imprecise exception ever does occur, it is a **fatal error**, so it should probably be crashing anyway.
e.g. use **evaluate** instead if you really need to catch.

```
---- Problem 1) DO NOT throw imprecise exceptions
--
-- BAD: try to avoid throwing imprecise exceptions
myDiv :: Int -> Int -> Int
```

```
myDiv _ 0 = throw DivideByZero    -- BAD. Imprecise
exception
myDiv x y = div x y

-- Solution 1a) use Maybe
myDiv :: Int -> Int -> Maybe Int
myDiv _ 0 = Nothing               -- GOOD
myDiv x y = Just $ div x y

-- Solution 1b) throw in IO
myDiv :: Int -> Int -> IO Int
myDiv _ 0 = throwIO DividByZero
myDiv x y = return $ div x y


---- Problem 2) DO NOT catch imprecise exceptions
--
x = 1 `div` 0                                      --
imprecise exception DivideByZero

main :: IO ()
main = do
  let handler e = print e >> return 0

  -- Solution 2a)
  return x                                        --
BEST. Avoid catching imprecise exception if possible
  catch @SomeException (return x) handler         --
BAD. May not catch imprecise exception.

  -- Solution 2b)
  -- use evaluate if you really need to catch an
imprecise exception
  -- evaluate: https://hackage.haskell.org/package/base/
docs/Control-Exception.html
  catch @SomeException (evaluate x) handler        --
OK. Catches DivideByZero.
  catch @ErrorCall (evaluate x) handler            --
OK. Catch imprecise only.
```

Basic syntax

```
---- Basic
--
import Control.Exception
data SomeException = forall e. Exception e =>
SomeException e
toException :: e -> SomeException
fromException :: SomeException -> Maybe e

-- SomeException
DivideByZero :: ArithException                    --
specific exception
SomeException DivideByZero :: SomeException       -- any
```

```
exception
toException DivideByZero :: SomeException         --
wrap in SomeException
handleError :: SomeException -> a
handleError se                                    --
unwrap SomeException
 | Just (TypeError e) <- fromException se = ...
 | Just DivideByZero <- fromException se = ...
 | Just StackOverflow <- fromException se = ...
 | ...
handleError (SomException e) = case e of          --
ERROR. Can't unwrap existential
  TypeError _ -> ...



catch :: Exception e => IO a -> (e -> IO a) -> IO a
catches :: Exception e => IO a -> [Handler a] -> IO a
catchJust :: Exception e => (e -> Maybe e') -> IO a ->
(e' -> IO a) -> IO a
```

```
-- Haskell
f :: IO a, onError :: Exception e => e -> IO a
f `catch` onError

-- analogous in JS
let result;
try { result = f(); } catch (e) { result = onError(e); }
return result;
```

```
f :: IO ()
f `catch` \(SomeException e) -> do
  putStrLn $ "error:" <> show e
  return ()

-- catching errors
f `catch` \(SomeException e) -> ...
  -- catch any exception
f `catch` \(e :: IOException) -> ...
     -- only IOExceptions
f `catches` [
  -- multiple exception types
  Handler $ \(e :: IOException) -> ... ,
  Handler $ \(e :: ArithException) -> ...
]
f `catch` \(SomeException se) -> do
-- similar
 case () of
  () | Just (e :: IOException) <- fromException e
-> ...
     | Just (e :: ArithException) <- fromException e
-> ...
(f `catches` \(e :: IOException) -> ...)
     -- BAD. Outer catch may catch error of inner
catch.
```

```
 `catches` \(e :: ArithException) -> ...)
    -- use catches instead.

-- handling errors
f `catch` \(SomeException e) -> throwIO e    -- rethrow
error
f `catch` \(SomeException e) -> pure ()      -- suprcess
error


-- handle: same as catch with args reversed
handle :: Exception e => (e -> IO a) -> IO a -> IO a
handleJust :: Exception e => (e -> Maybe e') -> (e' ->
IO a) -> IO a -> IO a
f :: IO a
f `catch` \(SomeException e) -> ...             -- catch
(\(SomeException e) -> ...) `handle` f          -- same
with handle


-- try: catch with error as Either
try :: Exception e => IO a -> IO (Either e a)
tryJust :: Exception e => (e -> Maybe e') -> IO a -> IO
(Either e' a)
f :: IO a
do
 res <- try f
 case res of
  Left (SomeException e) -> ...                -- error
  Right v -> ...                               --
success


-- bracket
finally :: IO a -> IO b -> IO a
onException :: IO a -> IO b -> IO a
bracket :: IO a -> (a -> IO b) -> (a -> IO c) -> IO c
bracket_ :: IO a -> IO b -> IO c -> IO c
bracketOnError :: IO a -> (a -> IO b) -> (a -> IO c) ->
IO c
bracket
 (putStrLn "Start" >> getResource)           -- setup
       getResource :: IO a
 (\r -> do                                   -- finally
       cleanup :: a -> IO b
   putStrLn "Done"
   cleanup r)
 (\r -> doSomething r )                      -- main
process   doSomething :: a -> IO c
```

```
-- e.g. bracket with file RW
import System.IO
bracket
  (hOpenFile "/tmp/test.txt" ReadWriteMode)
  hClose
```

```
  (flip hPutStr "Hello")
```

```
bracketOnError                                  -- same as
bracket, but finally only on error
 (putStrLn "Start" >> getResource)
 (\r -> do                                      -- only on
error
   putStrLn "ERROR"
   cleanup r)
 (\r -> doSomething r )

finally
 doSomething                                    -- main
process   doSomething :: IO a
 (putStrLn "Done" >> cleanup)                   -- called
both on error and success
onException                                     -- same as
finally, but only on error.
 doSomething
 (putStrLn "ERROR" >> cleanup)                  -- only on
error


import Control.Exception
```

==========

Asynchronous exception

Control.Exception: https://hackage.haskell.org/package/base/docs/Control-Exception.html#g:10

```
-- Control.Exception
data AsyncException = StackOverflow | HeapOverflow |
ThreadKilled | UserInterrupt
data SomeAsyncException = forall e. Exception e =>
SomeAsyncException e

asyncExceptionToException :: Exception e => e ->
SomeException
asyncExceptionFromException :: Exception e =>
SomeException -> Maybe e

throwTo :: Exception e => ThreadId -> e -> IO ()
mask :: ((forall a. IO a -> IO a) -> IO b) -> IO b
mask_ :: IO a -> IO a
uninterruptibleMask :: ((forall a. IO a -> IO a) -> IO
b) -> IO b
uninterruptibleMask_ :: IO a -> IO a

data MaskingState = Unmasked | MaskedInterruptible |
MaskedUninterruptible
```

```
getMaskingState :: IO MaskingState
interruptible :: IO a -> IO a
allowInterrupt :: IO ()
```

Asynchronous exception
Exceptions thrown into a target thread from a **different thread** to **interrupt** the target thread.
e.g. kill a thread, timeouts
Control.Concurrent.killThread tid = throwTo tid ThreadKillled
Async exception can **interrupt** the process anywhere.
e.g.

```
import Control.Exception
import Control.Concurrent

waitOneSecond = threadDelay $ 1000 * 1000
do
 thid <- myThreadId
 forkIO $ throwTo thid StackOverflow                --
throw error in main thread
 waitOneSecond                                      --
error interrupts during wait
  `catch`
  \(SomeException e) -> putStrLn $ "Caught error:" <>
show e
```

Masking
Ref: https://hackage.haskell.org/package/base/docs/Control-Exception.html#g:13
Masking is a mechanism to prevent interruption by async exceptions during **critical processes**
e.g. resource cleanup.
There are 3 levels of masking represented a **masking state**:
1) **Unmasked** state: may be interrupted by async exception any time
2) **MaskedInterruptible** state**:** not interrupted by async exception for **non-blocking** IO processes
non-blocking: e.g. IORef operations, tryTakeMVar/tryPutMVar (i.e. non-retry), forkIO
blocking: e.g. putStr, threadDelay, takeMVar/putMVar, etc
3) **MaskedUnInterruptible** state: not interrupted by async exception for **all** IO processes

Every builtin handler has a mask behind it
e.g. bracket: https://hackage.haskell.org/package/base/docs/src/GHC.IO.html#bracket

```
bracket
        :: IO a         -- ^ computation to run first
(\"acquire resource\")
        -> (a -> IO b)  -- ^ computation to run last
(\"release resource\")
        -> (a -> IO c)  -- ^ computation to run in-
between
        -> IO c         -- returns the value from the
```

```
in-between computation
bracket before after thing =
  mask $ \restore -> do
    a <- before
    r <- restore (thing a) `onException` after a
    _ <- after a
    return r
```

GOTCHA: finally never called because of async exception
e.g.

```
import Control.Exception
import Control.Concurrent

main = do
  td <- myThreadId

  bracket_
   (putStrLn "Start")
   (do                                          --
finally
     threadDelay $ 1000 * 1000
     putStrLn "Done")                           -- BAD.
Never called because of async exception
   (forkIO $ throwTo td StackOverflow)          -- throw
async exception
```

output

```
Start
```

Problem: mask in general only suspends **non-blocking IO** processes

Solution
Ref: https://www.fpcomplete.com/blog/2018/04/async-exception-handling-haskell/
Either
- wrap finally in **uniterruptibleMask**, or
- use **safe-exceptions** package which automatically wraps in uninterruptibleMask
  Control.Exceptions.Safe: https://hackage.haskell.org/package/safe-exceptions

```
start :: IO ()
done :: IO ()
process :: IO ()

-- Solution 1: use uninterruptibleMask
import Control.Exception
bracket_
  start
```

```
  (uninterruptibleMask_ done)
  process


-- Solution 2: use safe-exception
import Control.Exception.Safe
bracket_
 start
 done
 process
```

Syntax

```
---- Basics
--
import Control.Exception
import Control.Concurrent


-- throwTo: throw async exception
--
throwTo :: Exception e => ThreadId -> e -> IO a
do
 th <- myThreadId                       -- parent thread

 forkIO $ do
   throwTo th StackOverflow             -- throw in
parent thread
   ...                                    -- blocked
until thrown


-- mask: MaskedInterruptible state
-- i.e. non-interruptible during non-blocking IO
mask :: ((forall a. IO a -> IO a) -> IO b) -> IO b
mask_ :: IO a -> IO a
action :: IO a, cleanup :: e -> IO a
mask_ $ do                              -- always
MaskedInterruptible
  action `onException` cleanup
  cleanup
mask $ \restore -> do                   --
MaskedInterruptible except during restore
  (restore action)                      -- parent state.
i.e. usually Unmasked
    `onException` cleanup               --
MaskedInterruptible
  cleanup                               --
 MaskedInterruptible

mask_ $ do
  getMaskingState                          --
MaskedInterruptible
  th <- myThreadId
  forkIO $ throwTo th StackOverflow      -- Throw async
exception
```

```
  var <- newEmptyMVar                    -- nonblocking
i.e. not interrupted
  tryPutMVar var "a"                     -- nonblocking
  x <- tryTakeMVar  var

  putMVar var "a"                        -- blocking.
May be interrupted
  x <- takeMVar var
  putStrLn "Hello"                       -- blocking
  threadDelay 1000                       -- blocking
```

```
-- uninterruptibleMask: MaskedUnInterruptible state
-- i.e. non-interruptible during non-blocking IO
uninterruptibleMask :: ((forall a. IO a -> IO a) -> IO
b) -> IO b
uninterruptibleMask_ :: IO a -> IO a
action :: IO a, cleanup :: e -> IO a
uninterruptibleMask_ do
 getMaskingState                        --
MaskedInterruptible
 th <- myThreadId
 forkIO $ throwTo th StackOverflow      -- Throw async
exception

 putMVar var "a"                        -- blocking but
no async exception for all
 x <- takeMVar var
 putStrLn "Hello"                       -- blocking but
no e
 threadDelay 1000                       -- blocking
```